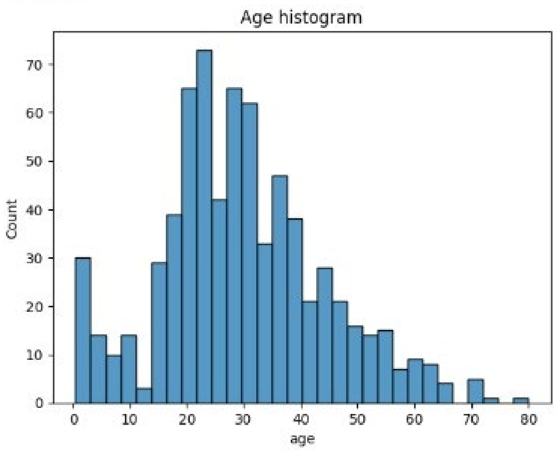
Day-6

```
[1]: #Load titanic dataset
import seaborn as sns
df=sns.load_dataset("titanic")
print(df.head())
```

```
   survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True
```

```
[3]: import matplotlib.pyplot as plt
sns.histplot(
    data=df,
    x="age",
    bins=30 #bins for accurate without bins it shows apprx val
)

plt.title("Age histogram")
plt.show()
```



Mean (Average)

[4]:
```
MARKS=[45,78,53,22,90]
Mean=sum(MARKS)/len(MARKS)
print("Mean:",Mean)
```

```
Mean: 57.6
```

Mode

[6]:
```
import statistics
data =[40,50,55,60,70]
mode=statistics.mode(data)
print("Mode:",mode)
```

```
Mode: 40
```

Median

[8]:
```
import statistics
marks=[40,50,60,70,70]
median=statistics.median(marks)
print("Median:",median)
```

```
Median: 60
```

[9]:
```
import pandas as pd
df =pd.DataFrame({
    "Marks": [45,50,55,60,75]
})
print("Mean:")
print(df["Marks"].mean())
print("\nMedian:")
print(df["Marks"].median())
print("\nMode:")
print(df["Marks"].mode())
```

```
Mean:
57.0

Median:
55.0

Mode:
0    45
1    50
2    55
3    60
4    75
Name: Marks, dtype: int64
```

Variance code

[11]:
```
import numpy as np
data=[10,12,14,16,18]
variance=np.var(data)
std=np.std(data)
print("Variance:",variance)
print("std:",std)
```

```
Variance: 8.0
std: 2.8284271247461903
```

IQR

[14]:
```
import numpy as np
data =[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR= Q3 -Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)
```

```
Q1: 13.75
Q3: 31.25
IQR: 17.5
```

[16]:
```
import numpy as np
data =[5,10,15,20,25,30,35,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR= Q3 -Q1
lower = Q1 -1.5 * IQR
upper = Q3 + 1.5 * IQR
print(lower)
print(upper)
outliers = [x for x in data if x < lower or x > upper]
print("Outliers:", outliers)
```

```
-12.5
57.5
Outliers: [100]
```

[17]:
```
sample_space={1,2,3,4,5,6}
event_a={2,4,6}
print("sample space(s):",sample_space)
print("event_a:",event_a)

p_a=len(event_a)/len(sample_space)
print("/npribability of event a")
print("p_a:",p_a)
```

```
sample space(s): {1, 2, 3, 4, 5, 6}
event_a: {2, 4, 6}
/npribability of event a
```

[19]:

```
import pandas as pd
df = pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,80]
})
corr = df['Hours'].corr(df['Marks'])
print("Pearson correlation:", corr)
```

```
Pearson correlation: 0.9933992677987827
```

[20]:

```
sample_space={1,2,3,4,5,6}
event_A={2,4,6}  #even numbers #P(A) is not in python so we put _

print("Sample Space(S):",sample_space)
print("Event A:",event_A)

#2.Probability

P_A=len(event_A)/len(sample_space)

print("\nProbability of Event A")
print("P(A) =",P_A)

#Conditional probability
#P(A|B)

event_B={4,5,6}    #Numbers greater than 3
intersection=event_A.intersection(event_B)
P_A_given_B=len(intersection) / len(event_B)

print("\nEvent B:",event_B)
print("A ∩ B:",intersection)
print("P(A|B)=", P_A_given_B)
```

```
Sample Space(S): {1, 2, 3, 4, 5, 6}
Event A: {2, 4, 6}

Probability of Event A
P(A) = 0.5

Event B: {4, 5, 6}
A ∩ B: {4, 6}
P(A|B)= 0.6666666666666666
```

[21]:

```
#$.Bayes Theorem code

#Ex:
#Disease prevalnce=1%
#test sensitivity=99%
#Positive test probability=5%

P_Disease=0.01
P_Positive_given_Disease=0.99
P_Positive=0.05

P_Disease_given_Positive=(
    P_Positive_given_Disease*P_Disease
```

```
#Positive test probability=5%

P_Disease=0.01
P_Positive_given_Disease=0.99
P_Positive=0.05

P_Disease_given_Positive=(
    P_Positive_given_Disease*P_Disease
)/P_Positive

print("\nBayes' Theorem Example")
print("P(Disease)=",P_Disease)
print("P(Positive|Disease)=",P_Positive_given_Disease)
print("P(Positive)=",P_Positive)

print(
    "P(Disease|Positive)=",
      round(P_Disease_given_Positive,4)
)
print(
    "Percentage=",
    round(P_Disease_given_Positive*100,2),
    "%")
```

```
Bayes' Theorem Example
P(Disease)= 0.01
P(Positive|Disease)= 0.99
P(Positive)= 0.05
P(Disease|Positive)= 0.198
Percentage= 19.8 %
```

[22]:

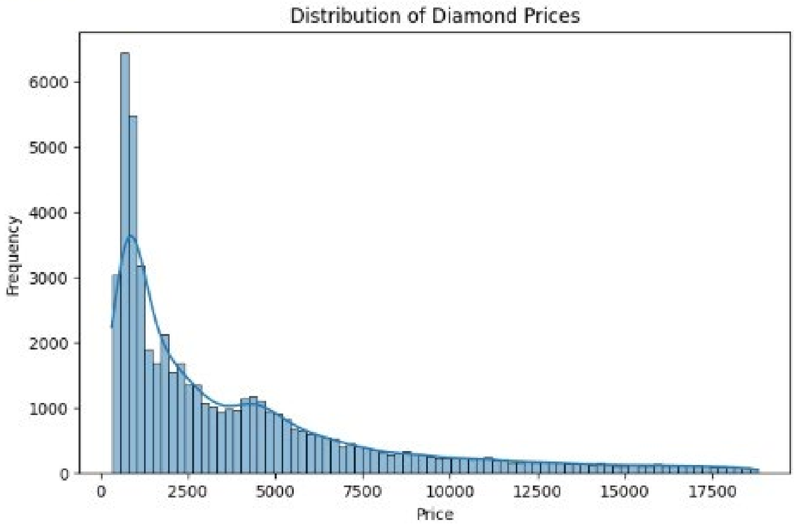
```
import pandas as pd
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,90]
})
corr=df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)

#close to +1
#strong positive relation
```

```
Pearson Correlation: 0.9948497511671097
```

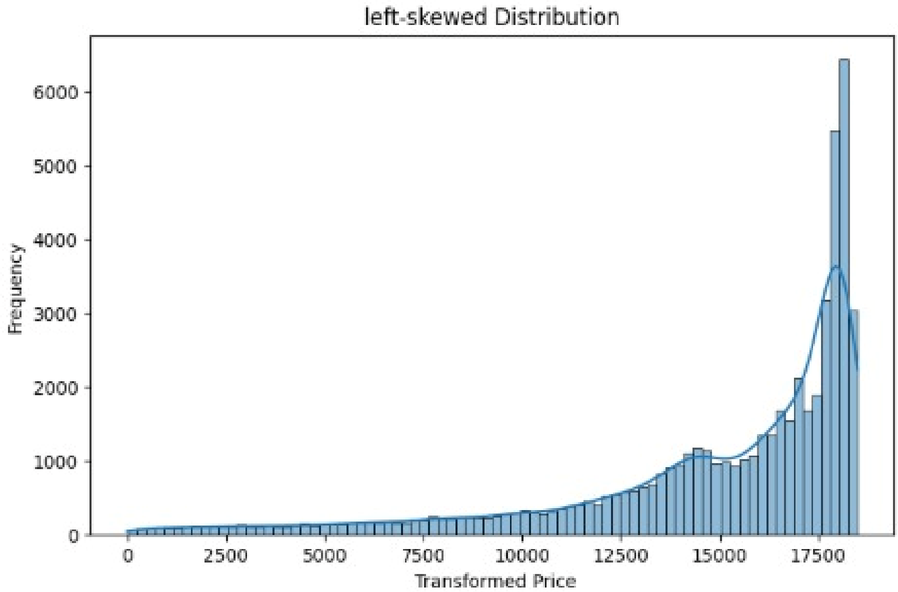
[26]:

```
import seaborn as sns
import matplotlib.pyplot as plt
df=sns.load_dataset("diamonds")
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)
plt.title("Distribution of Diamond Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```

```
plt.show()
```




```
[33]: import seaborn as sns
import matplotlib.pyplot as plt
df=sns.load_dataset("diamonds")
left_skew = df["price"].max()-df["price"]
plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)
plt.title(" left-skewed Distribution ")
plt.xlabel(" Transformed Price")
plt.ylabel("Frequency")
plt.show()
print("Skewness:",left_skew.skew())
```




```
Skewness: -1.6183952833835282
```

```
[ ]:
```